Performance Evaluation Assignment

1 Introduction

Firstly, I made the setup. This consisted of cloning the repo and installing tools and apps and other utilities such as mininet with other dependencies, iperf3, clang and also kitty and ryu or even ovs-test-controller, since I do not use them on a daily basis. I also tried to install everything in one go, therefore I also tried installing bpftune even though I am not certain whether it has been successfully installed, I went on solving the tasks. Since screenshots for proof of running are not needed, I just copy-pasted some output from shells.

2 Task 2.1

To find each container's IP address, I ran `ip a s` on each one and discovered that:

- h3 has 10.0.2.101
- h2 has 10.0.1.102
- h1 has 10.0.1.101

After this, I ran:

- on one h3: `iperf3 -s`
- on other h3: `iperf3 -s -p 5202` (since 5201 is the default port)
- on h1: `iperf3 -c 10.0.2.101`
- on h2: `iperf3 -c 10.0.2.101 -u -b 10M`

First I tried h1 and h3 (TCP) and then h1 and one h3 and after 2-3 seconds h2 and other h3 at the same time with h1 and h3. For running only TCP at the time, the throughput I found is:

```
[ID] Interval Transfer Bitrate Retr
[ 5] 0.00-10.00 sec 13.6 MBytes 11.4 Mbits/sec 0 sender
[ 5] 0.00-10.74 sec 11.0 MBytes 8.58 Mbits/sec receiver
```

I wanted to do this on the same h3 and at the same time, but it is not possible, since a port can be used once. The throughput I found while running TCP and UDP at the same time is:

For TCP from h1:

```
[ID] Interval Transfer Bitrate Retr
[ 5] 0.00-10.00 sec 6.25 MBytes 5.24 Mbits/sec 77 sender
[ 5] 0.00-11.40 sec 4.04 MBytes 2.98 Mbits/sec receiver
```

For UDP from h2:

```
[ID] Interval Transfer Bitrate Jitter Lost/Total Datagrams
[ 5] 0.00-10.00 sec 11.9 MBytes 10.0 Mbits/sec 0.000 ms 0/8632 (0%) sender
[ 5] 0.00-11.72 sec 11.6 MBytes 8.28 Mbits/sec 0.312 ms 258/8632 (3%) receiver
```

h3 TCP receiving:

```
[ID] Interval Transfer Bitrate
[ 5] 0.00-11.40 sec 4.04 MBytes 2.98 Mbits/sec receiver
```

h3 UDP receiving:

```
[ID] Interval Transfer Bitrate Jitter Lost/Total Datagrams
[ 5] 0.00-11.72 sec 11.6 MBytes 8.28 Mbits/sec 0.312 ms 258/8632 (3%) receiver
```

3 Task 2.2

Since the scripts are uploaded separately, I will include only the observations.

To compile and run the socket_diag script, I used the following commands:

```
gcc -o socket_diag socket_diag.c -lrt
sudo ./socket_diag /proc/$(pgrep -f "mininet:h1")/ns/net
```

Which resulted in output similar to:

```
sudo ./socket_diag /proc/$(pgrep -f "mininet:h1")/ns/net
================================
sport : 43944
dport : 5201
src ip : 10.0.1.101
dst ip : 10.0.2.101
inode : 1375030
Congestion window: 256
RTT: 285531 microseconds
Bytes acked: 385206
Unacked packets: 255
Retransmitted packets: 0
Total retransmits: 0
Delivery rate: 677431 bytes/s
Estimated throughput: 0.68 MB/s
================================
sport : 43942
dport : 5201
src ip : 10.0.1.101
dst ip : 10.0.2.101
inode : 1375029
Congestion window: 10
RTT: 215967 microseconds
Bytes acked: 143
Unacked packets: 0
Retransmitted packets: 0
Total retransmits: 0
Delivery rate: 6945 bytes/s
Estimated throughput: 0.01 MB/s
```

I realized that I had to run it multiple times manually second by second while running the iperf3 tests, so I automated the process with a small bash script. This collects a pair of data: the packet and the subsequent ACK. So, in 60 seconds, I collected 120 samples (double the expected amount).

After this, I needed to transform the collected data into a suitable format for analysis, so I created a small Python parser to convert it to a CSV file. This script produces output like:

```
connection_id,sport,dport,src_ip,dst_ip,inode,cwnd,rtt_us,bytes_acked,unacked,
    retrans,total_retrans,delivery_rate,throughput_mbps
10.0.1.101:45446-10.0.2.101:5201,45446,5201,10.0.1.101,10.0.2.101,657843,10,214484,165,0,0

10.0.1.101:45454-10.0.2.101:5201,45454,5201,10.0.1.101,10.0.2.101,657844,495,569875,731823
```

From these statistics and plots, I observed the following TCP phases:

TCP Phases Analysis:

1. **Slow Start Phase (0-5s):**
   - Exponential growth of cwnd: from 10 to 580
   - Throughput increases from 0.08 Mbps to 9.60 Mbps
2. **Congestion Avoidance Phase (5-20s):**
   - Congestion window grows linearly: from 10 to 10

- Average throughput: 4.52 Mbps

3. **Steady-State with AIMD (after 20s):**
   - Maximum congestion window: 2038
   - Maximum throughput: 9.60 Mbps
   - Total number of retransmissions: 0

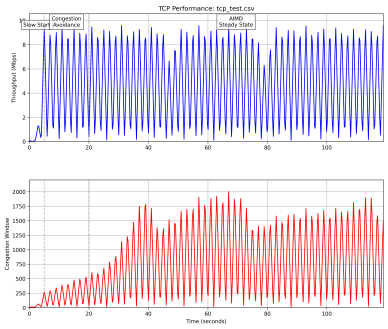


Figure 1: Description of your figure

4 Task 2.3

In order to do experiments, I had to slightly modify topology.py by adding arguments in the main section and links in the custom_topo method. I ran multiple tests manually (though this approach is less efficient, I wanted to ensure there were no running errors as my device is prone to errors). Though I wanted to create an automation script, I encountered issues like not seeing the namespace and others, so I opted for a manual (and time-consuming) approach. Nevertheless, it was a pleasant experiment. I used both kitty and xterm terminals for different parts of the experiment.

The procedure for each experiment was:

```
# <BW> is 5, 10, 20, 50
# <DELAY> is 0, 25, 50, 75, 100, 125 ms
sudo mn -c
sudo python3 topology.py --bw <BW> --delay <DELAY>
mininet> h3 xterm
mininet> h1 xterm
```

In another shell (not the one with mininet):

```
pgrep -f "mininet:h1"
pgrep -f "mininet:h3"
sudo mkdir -p /var/run/netns
sudo ln -sf /proc/pid_h1/ns/net /var/run/netns/h1
sudo ln -sf /proc/pid_h3/ns/net /var/run/netns/h3
sudo ip netns exec h3 kitty iperf3 -s
```

And in yet another shell:

```
sudo ip netns exec h1 xterm -T h1 -hold -e "bash -ic 'iperf3 -c 10.0.2.101 -p
    5201 -J -t 60 > /tmp/test.json; echo \"*** DONE ***\"; bash'"
sudo ip netns exec h1 cat /tmp/test.json > rbw_<BW>_d_<DELAY>.json
```

I performed this procedure for all bandwidth and delay combinations.

Plots and conclusions

Note: The graphs look somewhat unusual, which could be due to my device, Linux kernel version, or unstable internet connection. However, I've analyzed the results as follows:

Effect of change in bandwidth (Image 1)

- **Initial Volatility (0-10s):** All bandwidth configurations show high throughput fluctuations during the slow start phase, with peaks up to 20 Mbps regardless of configured bandwidth.

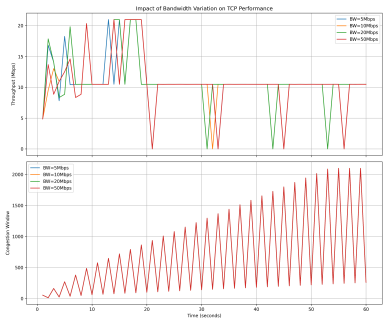


Figure 2: Description of your figure

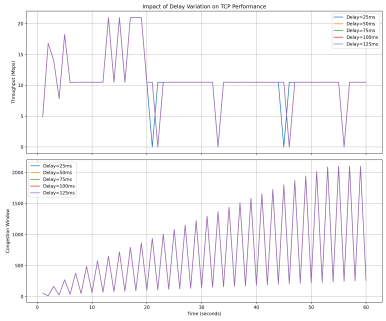


Figure 3: Description of your figure

- **Stabilization Pattern:** After initial volatility, throughput stabilizes around 10 Mbps for all configurations, with periodic drops to zero (likely packet loss events).
- **Congestion Window Behavior:** Only the 50Mbps configuration shows visible congestion window growth, with a classic sawtooth pattern increasing steadily from $\sim$100 to $\sim$2100 packets over time.

Delay Variation Impact (Image 2)

- **Dominant High Delay:** The 125ms delay configuration (purple) dominates both visualizations, suggesting other delay values have minimal impact on throughput and congestion window.
- **Throughput Behavior:** The 125ms configuration follows a similar pattern to the bandwidth graph - early spikes followed by steady 10 Mbps throughput with periodic drops.
- **Congestion Window Growth:** The 125ms configuration shows identical congestion window growth pattern to the 50Mbps case in the first graph, confirming the Bandwidth-Delay Product principle.
- **Performance Isolation:** The visualization suggests that at higher delays, TCP's congestion control algorithm becomes the dominant factor in performance rather than the actual network conditions.

Both graphs demonstrate TCP's adaptive behavior, with congestion window size growing steadily over time regardless of specific network parameters, and throughput stabilizing around similar values despite different configurations.

5 Task 2.4

For this task, I ran approximately the same procedure as in Task 2.1, but with bpftune enabled. Because my Linux kernel was outdated, I had to use a legacy version of bpftune without the BTF module (I attempted to update the kernel but was unsuccessful).

I ran an iperf3 server in h3 and a client in h1 using:

```
iperf3 -J -c 10.0.2.101 -t 60 > test1.json
```

Then, in another shell, I ran:

```
sudo bpftune -L -d -a tcp_conn_tuner -p 5201 --query > bpftune.log 2>&1
```

I then restarted mininet and the nodes and ran the same test again, saving the output to test2.json. I created a script to convert both JSON files to CSV format to create a comparative plot.

Strangely, the bpftune log was empty (possibly due to installation issues despite multiple attempts), which prevented me from analyzing the sysctls about buffers and packet sending/receiving evolution over time.

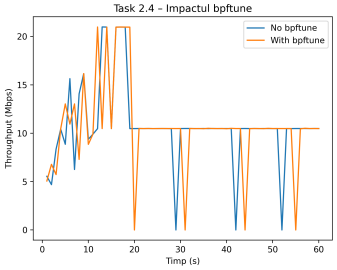


Figure 4: Description of your figure

Analysis of bpftune impact

The plot shows the impact of bpftune on TCP performance:

- **Similar overall performance:** Both configurations (with and without bpftune) reach similar throughput peaks (approximately 21 Mbps) and stabilize around 10 Mbps.
- **Performance drops:** Both configurations experience complete temporary throughput drops (to 0 Mbps), likely representing moments of congestion and packet loss.
- **Main bpftune benefit:** With bpftune enabled (orange line), performance drops are less frequent than without bpftune (blue line).
- **Improved stability:** After the initial period (>20s), the bpftune configuration maintains a more stable 10 Mbps throughput with fewer interruptions.

In conclusion, bpftune improves TCP connection stability by automatically adjusting network parameters, reducing the frequency of congestion events and recovering more efficiently from packet loss situations.

6 General Conclusion

During this assignment, I have discovered several key insights about TCP performance:

- **Adaptive Convergence:** Despite varying network conditions, TCP adapts to converge on roughly the same levels of throughput due to its congestion control mechanisms (Figure **??**).
- **BDP Principle Observed:** Higher delay requires wider congestion windows for sustaining throughput, confirming the Bandwidth-Delay Product theory.
- **Performance Paradox:** Counterintuitively, systems with higher bandwidth do not necessarily deliver better performance due to TCP's self-throttling behavior.
- **Automatic Tuning Value:** BPFtune improves stability and reduces throughput interruptions by dynamically optimizing system parameters based on observed network conditions.

These results indicate that TCP performance optimization requires understanding the protocol's inherent adaptations rather than simply pursuing higher bandwidth or lower delay. The efficiency of congestion control algorithms and recovery mechanisms ultimately determines the user experience in real-world network environments.